Python Engineering - enumerate() + Error Position Guide

Iterar archivos, numerar líneas y localizar datos corruptos en la futura matriz NumPy

1. ¿Por qué ponemos file dentro de enumerate()?

Dentro de `with open(... ) as file`, la variable `file` es un **objeto archivo abierto**. No es la ruta: es el objeto que permite leer su contenido.

```
with open(file_path, 'r', encoding='utf-8') as file:
    ...
```

Ese objeto es **iterable**: Python puede pedirle una línea, luego la siguiente, y así hasta el final.

```
for line in file:
    print(line)
```

Por eso podemos usarlo como primer argumento de enumerate():

```
for line_number, line in enumerate(file, start=2):
    ...
```

enumerate() necesita algo que pueda recorrer. En este caso, ese iterable es el propio archivo abierto.

2. file_path y file no son lo mismo

Variable	Qué representa	Ejemplo
file_path	Dónde está el archivo	C:/project/data/file.csv
file	Objeto abierto para leer/escribir contenido	devuelto por open(...)

No hacemos `enumerate(file_path)` porque eso recorrería la representación de la ruta, no las líneas de contenido que queremos procesar.

3. Qué ocurre tras readline()

```
header = file.readline().strip()
```

Esa llamada consume la primera línea. El objeto archivo recuerda internamente su posición actual.

```
line 1 -> h_m,v_m_s,t_s      # already consumed
line 2 -> 100.0,20.0,0.0      # next line
line 3 -> 102.0,BAD,0.1
```

Entonces `enumerate(file, start=2)` empieza a numerar la primera línea restante como 2.

4. start=2 solo cambia el contador

```
items = ['A', 'B', 'C']

for n, item in enumerate(items, start=2):
    print(n, item)

# 2 A
# 3 B
# 4 C
```

No se saltan A ni B. El primer elemento sigue siendo A; simplemente su contador empieza en 2.

5. ¿Qué produce enumerate(file, start=2)?

```
(2, '100.0,20.0,0.0\n')
(3, '102.0,BAD,0.1\n')
(4, '103.9,18.0,0.2\n')
```

El for desempaqueta cada pareja en `line_number` y `line`.

6. Tres posiciones diferentes: no mezclarlas

Cuando queremos diagnosticar un dato corrupto, podemos hablar de tres coordenadas distintas:

Coordenada	Ejemplo	Significado
Línea física del CSV	line_number = 3	Tercera línea real del archivo, contando cabecera
Fila lógica de datos	row_index = 1	Segunda fila de datos, usando índice Python desde 0
Columna del array	column_index = 1	Segunda columna: velocidad

Para el valor corrupto de la línea física 3, la posición futura en una matriz [h,v,t] sería normalmente `[1, 1]`.

7. Importante: si float() falla, todavía no existe esa celda en el np.array final

En nuestro loader de EX23 construimos primero una lista:

```
rows = []
...
rows.append([h, v, t])
...
return np.array(rows)
```

Si falla `float('NOT_A_NUMBER')`, la fila corrupta **no llega a añadirse** a rows. Por tanto, técnicamente no existe todavía una posición ocupada en el np.array final.

Lo que sí podemos informar es la **posición que habría ocupado** si la conversión hubiese sido válida.

8. Forma simple si sabemos qué columna estamos convirtiendo

En el código actual sabemos que la velocidad es la columna 1:

```
row_index = len(rows)

try:
    v = float(values[1])

except ValueError as error:
    raise ValueError(
        f'Line {line_number}: invalid velocity '
        f'at array position [{row_index}, 1]: '
        f'{values[1]!r}'
    ) from error
```

¿Por qué `row_index = len(rows)`? Porque rows contiene exactamente las filas válidas que ya hemos guardado. La siguiente fila válida ocuparía ese índice.

9. Ejemplo concreto

```
rows currently contains:
[[100.0, 20.0, 0.0]]

len(rows) -> 1
line_number -> 3
values[1] -> 'NOT_A_NUMBER'

would-be NumPy position -> [1, 1]
```

Podemos obtener un mensaje como:

```
Line 3: invalid velocity at array position [1, 1]: 'NOT_A_NUMBER'
```

10. Forma más general: localizar cualquier columna corrupta

Si queremos evitar tres bloques separados para h, v y t, podemos recorrer también las columnas con enumerate():

```
column_names = ['h_m', 'v_m_s', 't_s']
numeric_row = []
row_index = len(rows)

for column_index, value in enumerate(values):

    try:
        numeric_value = float(value)

    except ValueError as error:
        raise ValueError(
            f'Line {line_number}, '
            f'array position [{row_index}, {column_index}], '
            f'column {column_names[column_index]}: '
            f'invalid value {value!r}'
        ) from error

    numeric_row.append(numeric_value)

rows.append(numeric_row)
```

Aquí usamos enumerate() dos veces con dos propósitos distintos:

enumerate	Qué numera
enumerate(file, start=2)	Líneas físicas del archivo
enumerate(values)	Columnas de la fila; empieza en 0

11. ¿Qué significa 'from error'?

`raise ... from error` conserva la relación entre el error original de float() y nuestro nuevo mensaje más informativo.

```
float('NOT_A_NUMBER')
    -> original ValueError

raise ValueError('Line 3 ... [1,1] ...') from error
    -> engineering-friendly ValueError with original cause preserved
```

No es obligatorio para completar EX23, pero es una práctica útil cuando transformamos una excepción en otra más descriptiva.

12. ¿Y si hay líneas en blanco?

Aquí aparece una razón por la que `row_index = line_number - 2` no siempre es robusto.

```
line 1 -> header
line 2 -> valid row
line 3 -> blank line (skipped)
line 4 -> corrupt row
```

La línea física sería 4, pero si saltamos la línea vacía la futura fila NumPy seguiría siendo índice 1. Por eso `len(rows)` describe mejor el índice lógico de la matriz que `line_number - 2`.

13. Si el array ya estuviera preasignado

En una estrategia de preasignación sí existiría físicamente una matriz antes de convertir todos los datos:

```
trajectory = np.empty((n_rows, 3))
trajectory[row_index, column_index] = value
```

En ese caso podemos hablar literalmente de una posición existente. En nuestro loader actual, hablamos de la **posición prevista** porque construimos el array solo al final.

14. Comparación: línea CSV vs posición NumPy

```
CSV
line 1 -> header
line 2 -> data row 0 -> NumPy row 0
line 3 -> data row 1 -> NumPy row 1
line 4 -> data row 2 -> NumPy row 2

Columns:
h -> column 0
v -> column 1
t -> column 2
```

Si no hay líneas omitidas, la relación parece sencilla. Aun así, separar **line_number** y **row_index** hace el programa más claro y resistente.

15. Chuleta final

```
file_path
-> location of the file

file
-> open file object
-> iterable line by line

enumerate(file, start=2)
-> physical file line number + line text

row_index = len(rows)
-> next logical NumPy row index

enumerate(values)
-> column index + text value

ERROR LOCATION EXAMPLE
line_number = 3
row_index = 1
column_index = 1
-> future array position [1, 1]
```

Idea central: el número de línea sirve para encontrar el problema en el archivo; la pareja `[row_index, column_index]` sirve para expresar dónde habría aparecido dentro de la estructura numérica.